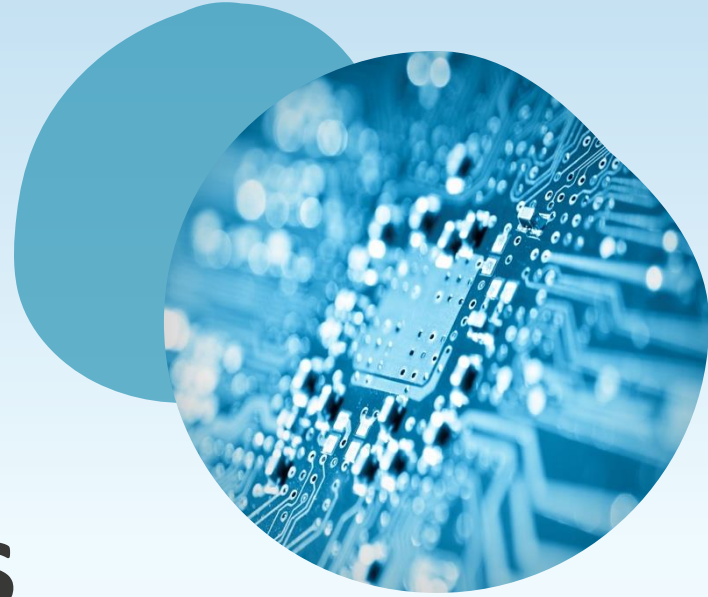


Introduction to embedded systems



Embedded Software for the Internet of Things

Kasım Sinan Yıldırım – University of Trento

Course at a glance

- Open to students in Computer Science and ICE programs
- Embedded software for connected, resource-constrained devices
- Theory and hands-on work develop in parallel
- The course begins with embedded-system fundamentals

Who am I?

- Associate Professor@DISI, University of Trento
 - **Office: 120 (Povo 2)**
- Research focus: embedded systems and sustainable sensing
 - *Where the theory can meet the practice*
- Course perspective:
 - *Connect theory with working systems*



Call me
"Sinan".

Research Context

- Sensing that keeps going
 - *Tiny embedded devices can monitor the physical world for years with very little maintenance.*

EnergyHarvesting

Environmental Sensing Wearables

Batteryless Computing



*Future sensing devices are tiny, sustainable, **smart**, and run forever!*

Learning outcomes

- Learn the fundamentals to design and develop IoT applications:
 - programming languages
 - hardware platforms
 - operating systems
 - software techniques, platforms and tools

Learning outcomes

- Interface software with modern **microcontroller** platforms
 - Acquire the **theoretical and practical skills** to interface with state-of-the-art embedded computing platforms.
- Design **embedded applications**
 - Under constraints, e.g., timing and energy
- Use peripherals like timers and ADCs
 - Others, like communication interfaces and operating-system services
- Build and demonstrate **a complete IoT application**
 - Learn the fundamentals to design and develop IoT applications:
 - programming languages, hardware platforms, operating systems, software techniques, platforms and tools

Recommended background

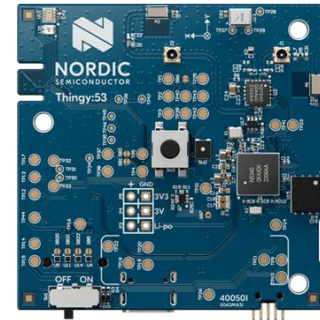
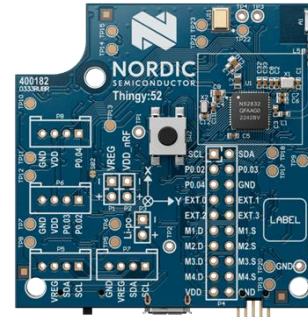
- No formal prerequisite is required:
 - **C programming** and **software design**
- Background on the following topics is preferable:
 - Basic computer architecture and operating systems
 - ARM assembly is useful but not essential
 - Introductory electronics

Course Roadmap

- Development Environments for Embedded Systems
- Embedded Software Design, Implementation and Testing
- Microcontroller-based IoT Platforms
 - Texas Instruments MPS432
- Real-time scheduling and embedded realtime operating systems
- IoT Application Architecture

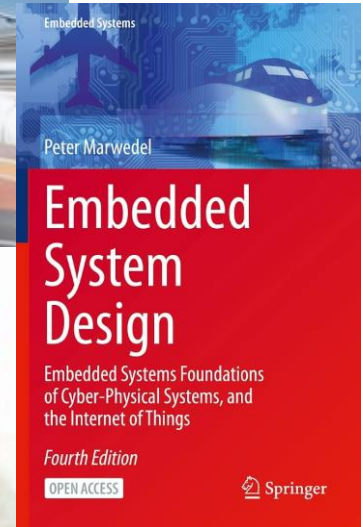
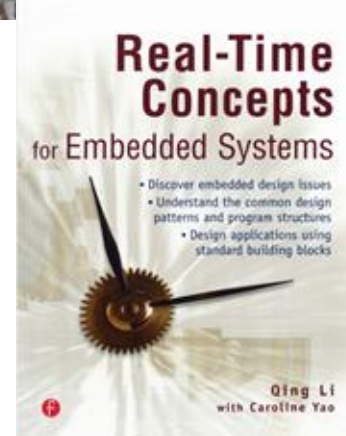
Course Roadmap

- Bare Metal Programming
 - Microcontroller Interfacing, Ports and I/O
 - Polling/Interrupt Management
 - Timers and timer services
 - Analog to Digital Converters (ADC), Direct Memory Access (DMA)
 - Serial Communication (UART), SPI, I2C
- Sensor interfacing
 - Accelerometer, temperature, light, ...
- Application examples
- Programming Embedded RTOS



Course Materials

- The course website contains slides, supporting material, and recorded lectures
 - <https://didatticaonline.unitn.it/dol/user/view.php?id=102360&course=44590>
- Several books as supplemental text:
 - *Computers as Components: Principles of Embedded Computing System Design*, Marilyn Wolf
 - *Real-Time Concepts for Embedded Systems*, Qing Li, Caroline Yao, Taylor & Francis Ltd
 - *Embedded System Design, Embedded Systems Foundations of Cyber-Physical Systems, and the Internet of Things*, Marwedel, P., Springer



Teaching Assistants

- Renan Beran Kilic renanberan.kilic@unitn.it
- Jan Leuken jan.lueken@unitn.it
- Use Teacher-student discussion forum **for all Q&A:**

[Courses](#) / [2026/2027](#) / [Dipartimento di Ingegneria e Scienza dell'Informazione](#) / Embedded Software for the Internet of Things [145996] - YILDIRIM

Embedded Software for the Internet of Things [145996] - YILDIRIM

Course

Settings

Participants

Grades

Reports

More ▾



General



Notice board

Teachers publish news and information regarding the course. Students cannot reply to messages.



Teacher-student discussion forum

Forum for the interaction between students and teacher.

Lecture Format

- We will have concept-oriented lectures to establish the theory
- Practice lectures work through code and applications
- You can bring your laptop to try the examples during the class.
 - I will share the hardware kits during class.

Assessment

- 30-35% Written Exam (Theory)
- 65-70% Project Evaluation
- You will build a **team** to design, implement, and demonstrate **an IoT application**
 - Deliverables include source code, instructions, and a demo video



Build something people remember.

Selected Projects (2025-2020)

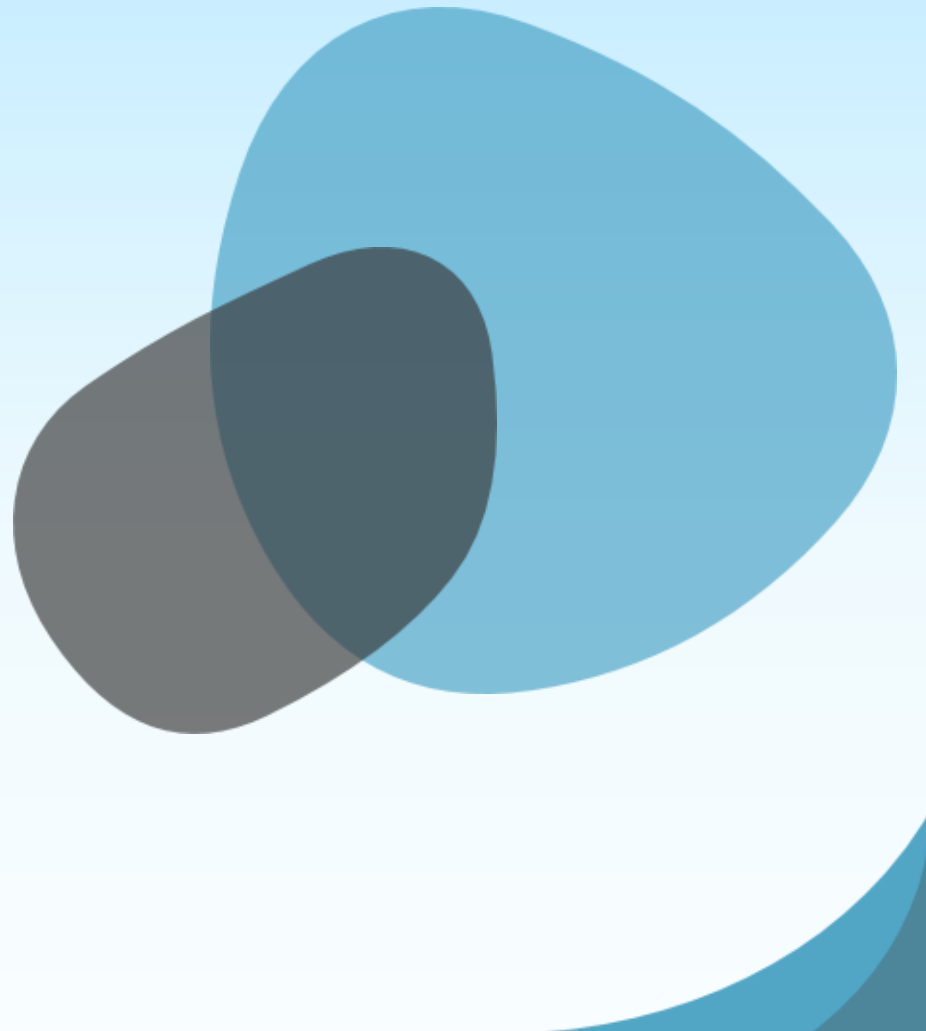
Last Year

- [Versy - Liquid Pouring Robot](#)
- [RedPanda4x4](#)
- [Orione](#)
- [IntelligentVase](#)
- [SUNFLOWER](#)
- [BTFL-Mouse](#)
- [GrooveChip](#)

2024-2020

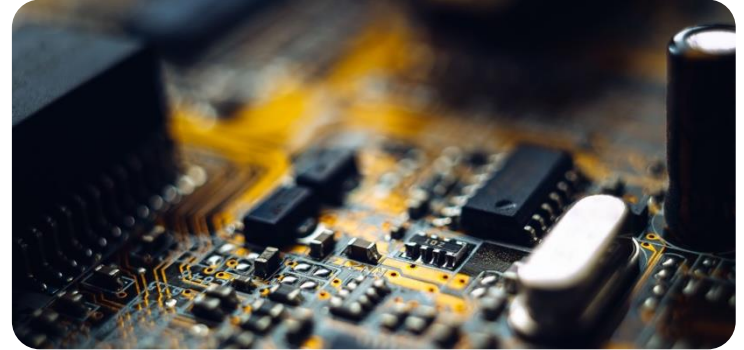
- [Konfort](#)
- [Smart MSP](#)
- [Claw Machine](#)
- [Sun Scanner](#)
- [Oscilloscope](#)
- [Receiver & Transmitter on RC-CAR](#)
- [Cube Solver](#)

Introduction



Embedded Systems: Definition

- Computing systems with **tightly coupled** hardware and software integration:
 - Designed to perform a dedicated functionality
- The word “**embedded**” means:
 - Built into (to be an integral part of) a larger system
 - The larger system is called the embedding system.

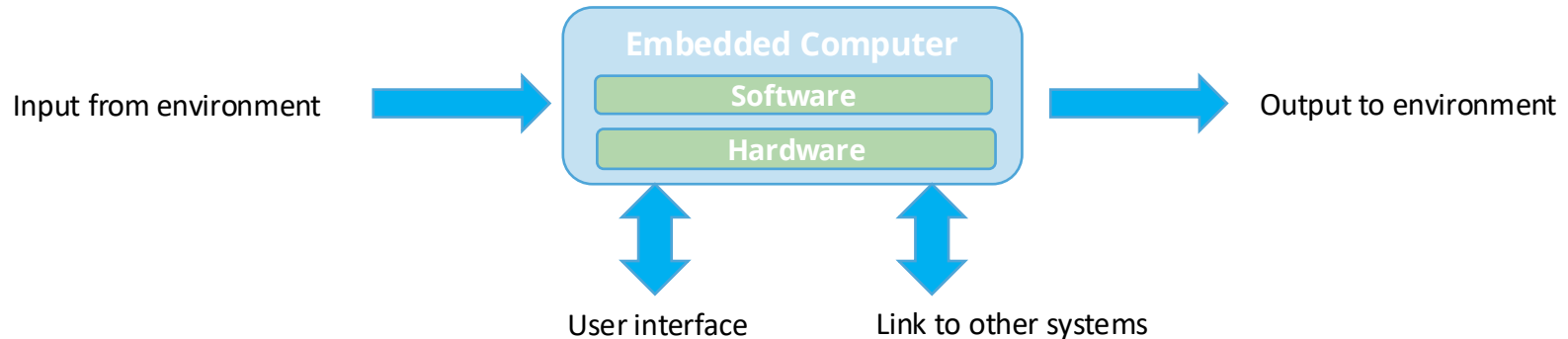


*Hardware and software work together to perform a **dedicated function** inside a larger system.*

Embedded Systems: Definition

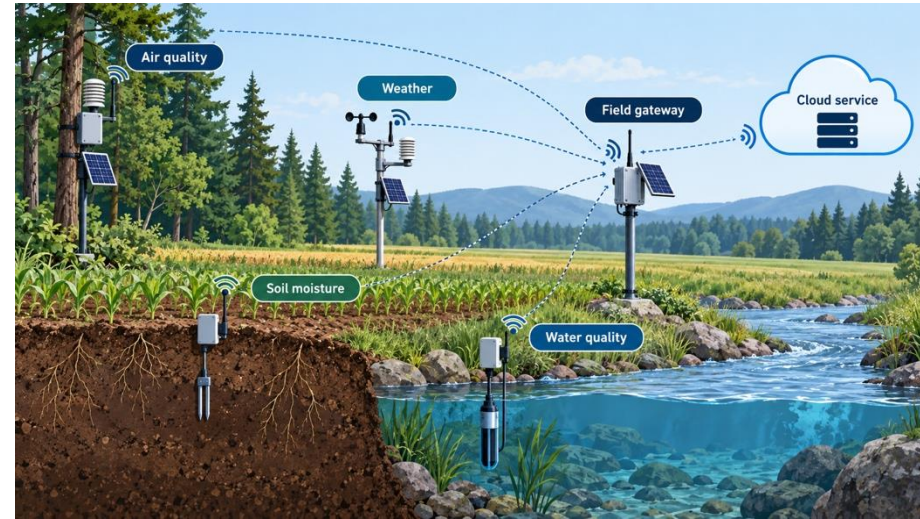
- **Application-specific** computer system
 - Many embedded systems must also respond within a defined time (**realtime constraints**).

Environment	Embedded computer	Environment
Sensors and user input	Hardware + software	Actuators, display, network



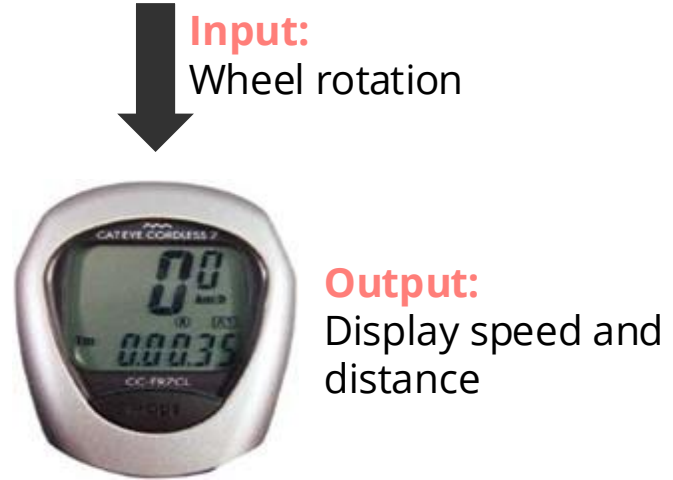
Why embed computing?

- Lower cost and more performance
 - Application-specific and optimized system
- Improve dependability
 - One device fails but others continue
- More functionality
 - E.g., intelligence
- Scalability



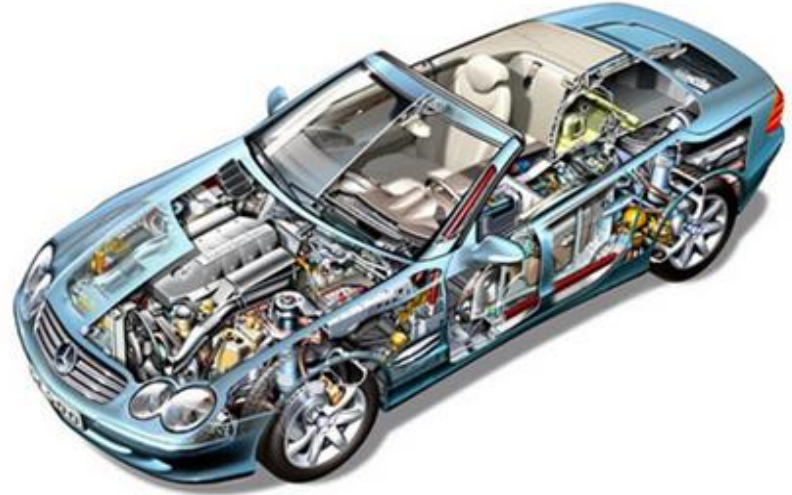
Bike computer example

- Wheel rotation becomes speed and distance on a local display.
- The design is shaped by **size, cost, weight, power, and energy**.
- A small 8-bit **microcontroller (MCU)** can be enough.
 - What is a microcontroller?



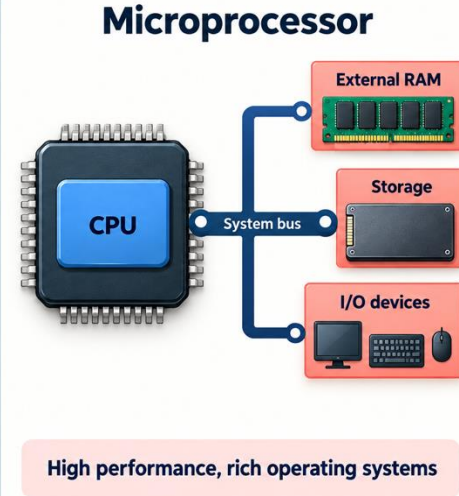
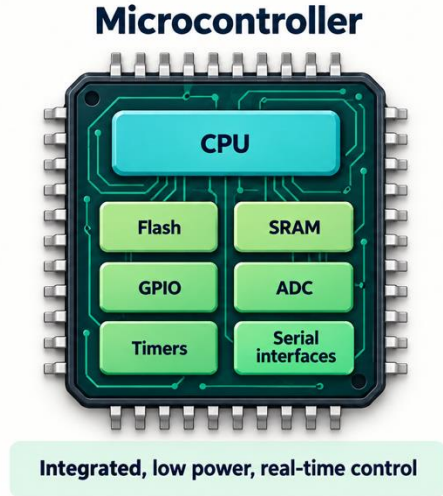
A vehicle is a network of computers

- Today's high-end automobile may have **100 processors**:
 - Many specialized controllers
 - 4-bit microcontroller checks seat belt;
 - Microcontrollers run dashboard devices;
 - 16/32-bit microprocessor controls the engine.
 - Low-end cars use 20+ processors.
- Different timing and safety needs



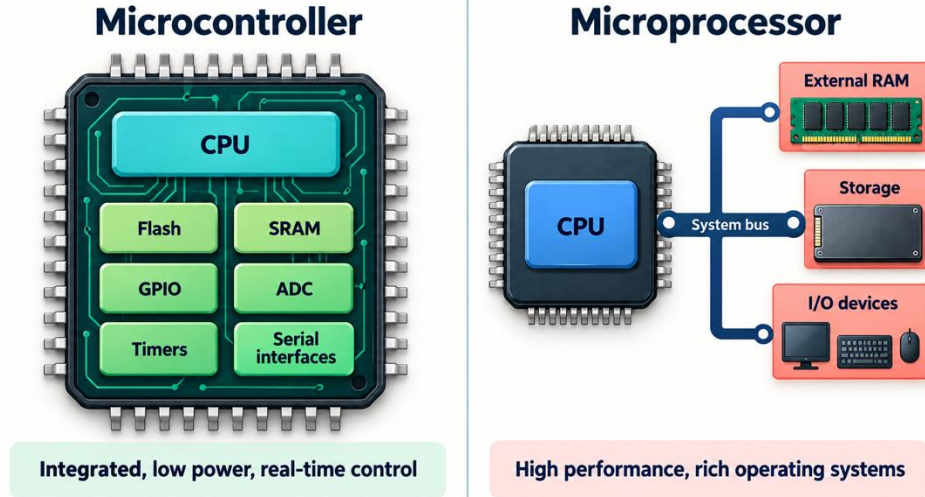
Microcontroller vs. Microprocessor

- What is the difference between an MCU and a CPU?
 - Both have a core to execute instructions
 - Both devices fetch, decode, and execute instructions
 - Both contain registers and an arithmetic logic unit



Microcontroller vs. Microprocessor

- What is the difference between an MCU and a CPU?
 - Both have a core to execute instructions
 - Both devices fetch, decode, and execute instructions
 - Both contain registers and an arithmetic logic unit

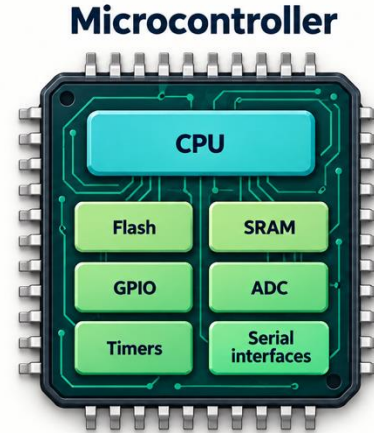


The main distinction lies in **what surrounds** the processor core

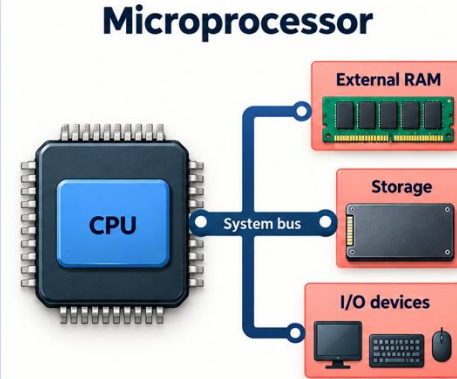
Microcontroller vs. Microprocessor

- **CPU (Microprocessor)**

- A single processor core that supports at least instruction fetching, decoding, and executing
- Used for general-purpose computing but depends on external memory and peripherals.



Integrated, low power, real-time control



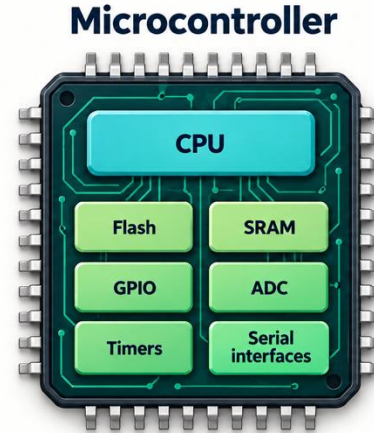
High performance, rich operating systems

The main distinction lies in **what surrounds** the processor core

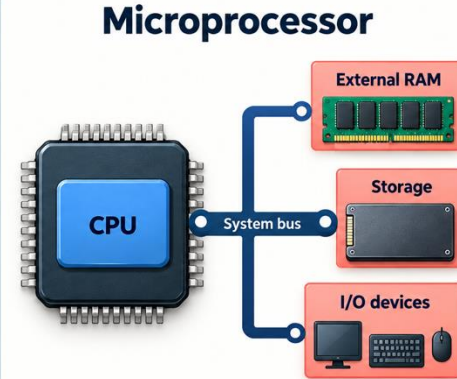
Microcontroller vs. Microprocessor

- **MCU (Microcontroller)**

- Typically has a single processor core
- Typically used for basic control purposes
- Has memory blocks, Digital IOs, Analog IOs, and other basic peripherals



Integrated, low power, real-time control

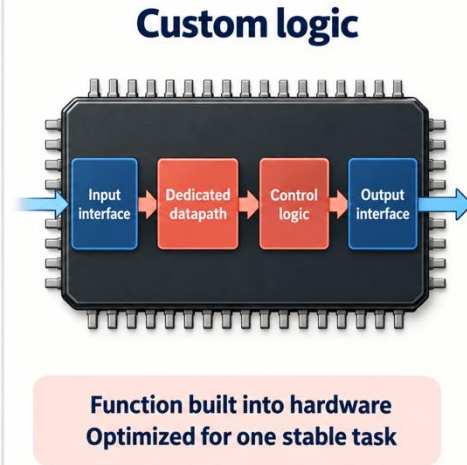
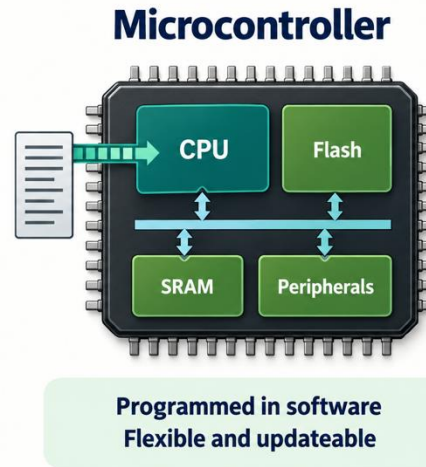


High performance, rich operating systems

An MCU **integrates** the processor, memory, and common peripherals **on one chip**.

Dedicated hardware

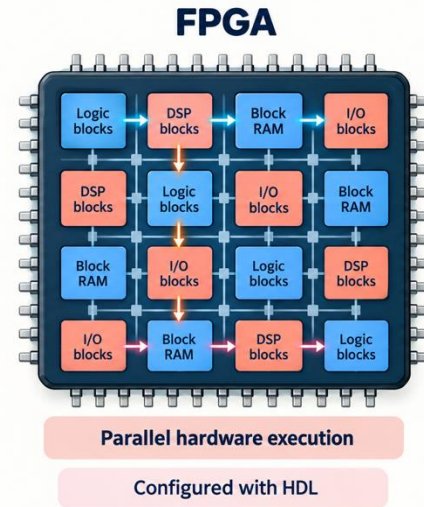
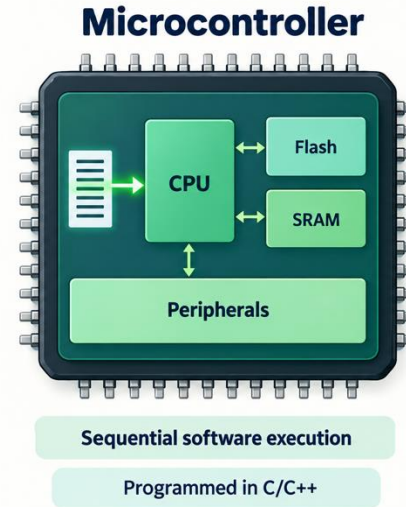
- **Custom logic** implements a narrow, well-defined function
 - High performance or energy efficiency
 - Updates are harder



The value of programmability: The same hardware can perform different functions by changing only the software.

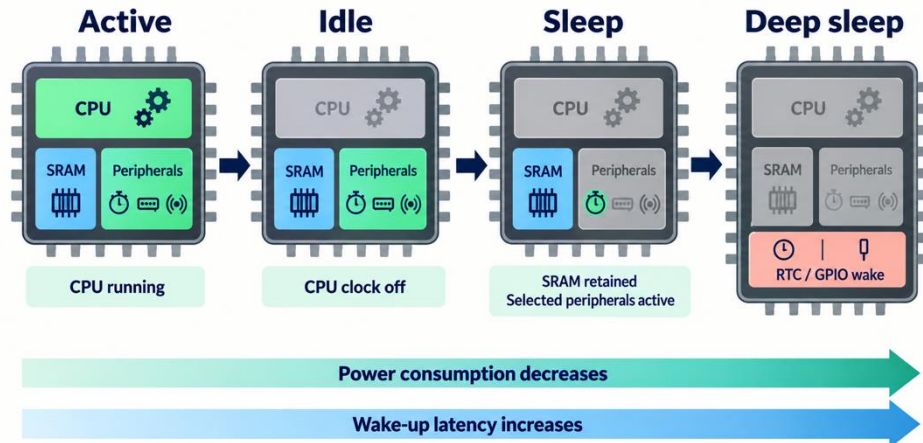
Alternatives

- Field-programmable gate arrays (FPGAs), custom logic, etc.
 - FPGAs offer reconfigurable logic between fixed hardware and software.



Why MCUs fit embedded products

- Low development and manufacturing cost
 - Integrated peripherals reduce board complexity
- Enough performance for sensing and control
- Low power consumption and effective sleep modes



Options for Building Embedded Systems

		Implementation	Design Cost	Unit Cost	Upgrades & Bug Fixes	Size	Weight	Power	System Speed
Software Running on	Dedicated Hardware	Discrete Logic	low	mid	hard	large	high	?	very fast
		ASIC	high (\$500K/mask set)	very low	hard	tiny - 1 die	very low	low	extremely fast
		Programmable logic – FPGA, PLD	low	mid	easy	small	low	medium to high	very fast
	Generic Hardware	Microprocessor + memory + peripherals	low to mid	mid	easy	small to med.	low to moderate	medium	moderate
		Microcontroller (int. memory & peripherals)	low	mid to low	easy	small	low	medium	slow to moderate

Resource constraints

- How much hardware do we need?
 - How fast is the CPU? Memory size?
- How do we meet our deadlines?
 - Faster hardware or cleverer software?
- How do we minimize power?
 - Turn off unnecessary logic? Reduce memory accesses?

Correctness, evolution, and security

- Does it really work?
 - Is the specification correct?
 - Does the implementation meet the specifications?
 - How do we test for real-time characteristics?
 - How do we test on real data?
- How do we design for upgradeability?
 - How to add features by changing software and hardware?
- Is it secure?
 - How are the dangerous acts prevented?

Why embedded testing is difficult?

- Complex testing:
 - We cannot separate the testing of an embedded computer from the machine in which it is embedded.
 - How to test a washing machine software?
- Limited observability and controllability:
 - Difficult to see what is going on
 - no keyboards and screens, need to watch the values of electrical signals
- Restricted development environments:
 - Much more limited than those available for PCs

Core properties

- Interfacing with the environment:
 - **Analog signals** from sensors: e.g., voltage value that represents a physical value
- **Concurrent** and **reactive** behaviours:
 - Respond to events **timely** (real-time constraints on responses)
 - Must perform multiple separate activities **concurrently** (e.g., several sensors...)
- Fault handling
 - Operate independently for long periods and handle likely faults without crashing
- Diagnostics
 - Help developers determine problems quickly

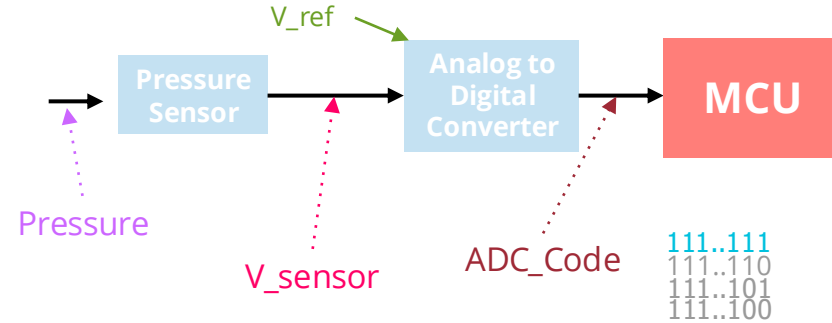
Example: Analog sensing path

- We calculate how deep an underwater object is:
 - A sensor detects **pressure** and generates a proportional **output voltage V_{sensor}**
 - Analog to Digital Converter generates a proportional digital integer (code) based on **V_{sensor}** and **V_{ref}**

Physical quantity	Sensor voltage	ADC code	Software value
Pressure	Proportional analog signal	Integer sample	Calibrated pressure

Example: Analog sensing path

- We calculate how deep an underwater object is:
 - A sensor detects **pressure** and generates a proportional **output voltage V_{sensor}**
 - Analog to Digital Converter generates a proportional digital integer (code) based on **V_{sensor}** and **V_{ref}**

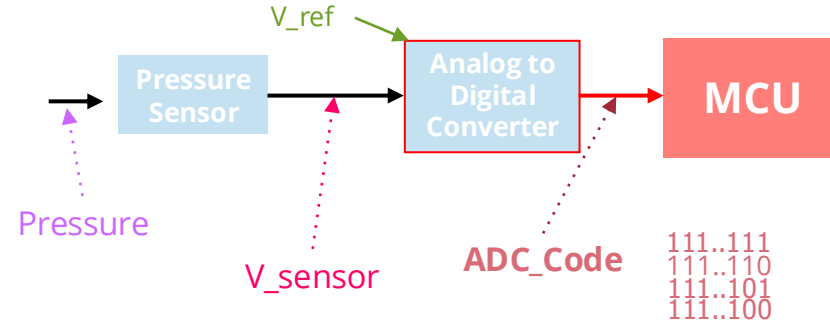


From ADC code to voltage

- The **Analog-to-digital Converter (ADC)** converts a voltage range into **integer codes**

- Reference voltage and ADC resolution define the conversion scale
- Clipping occurs when the input exceeds the supported range
- MCU reads these integer codes.

$$V_{\text{sensor}} = \text{ADC_code} \times V_{\text{ref}} / \text{ADC_max}$$



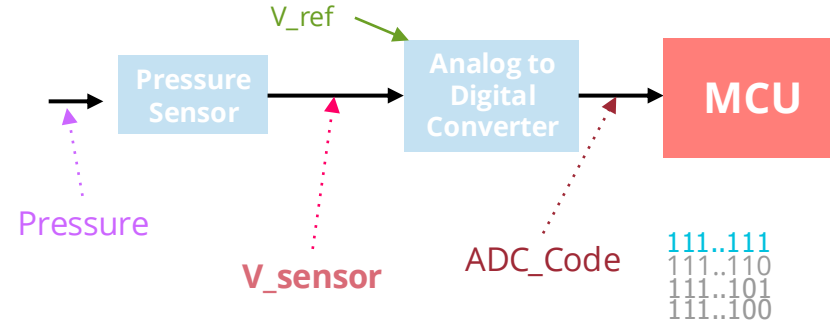
Software on MCU

```
// Your Software  
ADC_Code = adc_read();
```

From ADC code to voltage

- The MCU software should **convert** the measured integer code back into the voltage reading.

$$V_{\text{sensor}} = \text{ADC_code} \times V_{\text{ref}} / \text{ADC_max}$$

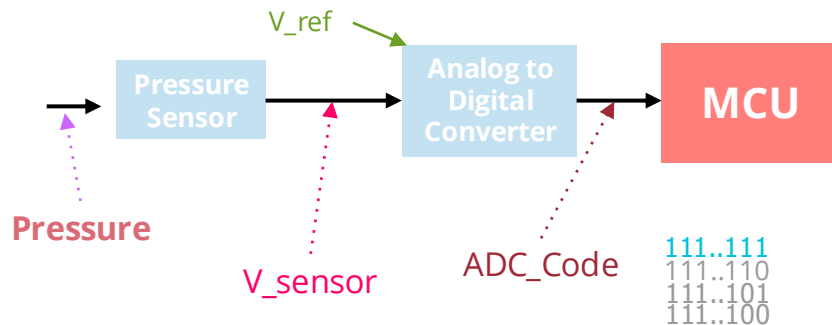


Software on MCU

```
// Your Software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
```

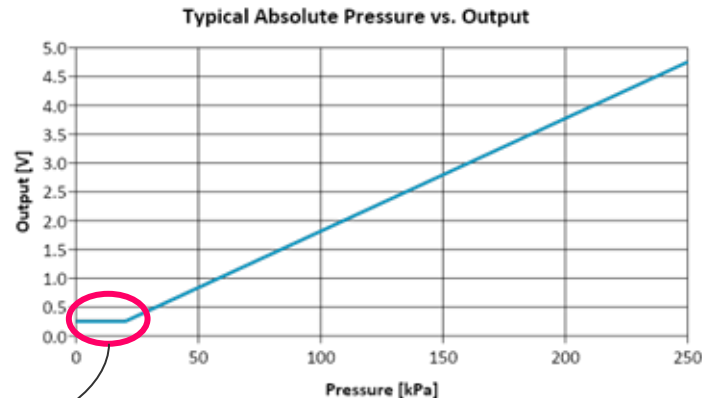
From voltage to pressure

- The **sensor model** converts voltage into pressure
 - Calibration constants, units, and environmental assumptions must remain explicit



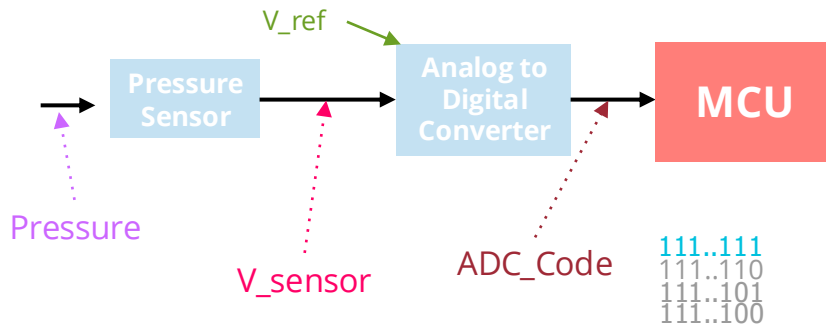
Software on MCU

```
// Your Software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / V_supply + 0.04);
```



From pressure to depth

- A second model **converts** pressure difference **into depth**
 - Calibration constants, units, and environmental assumptions must remain explicit
 - Each conversion adds uncertainty that software must manage



$$\text{depth} = f(\text{pressure} - \text{atmospheric pressure})$$

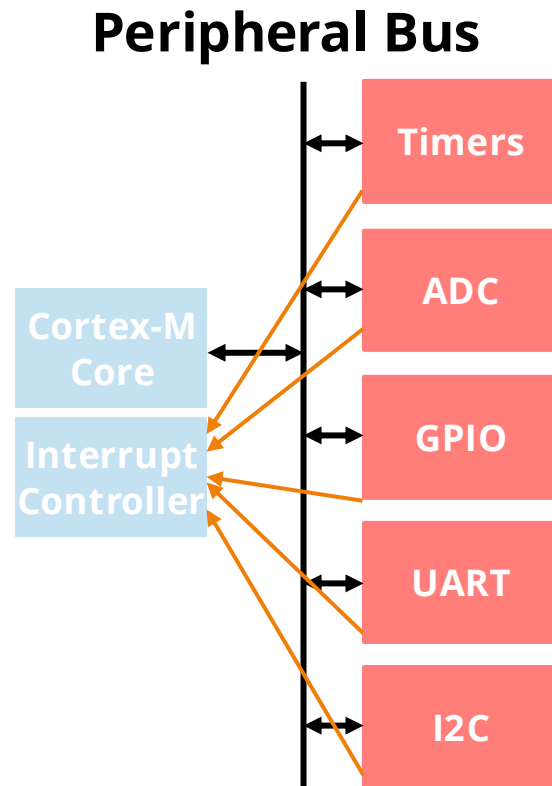
Software on MCU

```
// Your Software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / V_supply + 0.04);
Depth_ft = 33 * (Pressure_kPa - Atmos_Press_kPa) / 101.3;
```

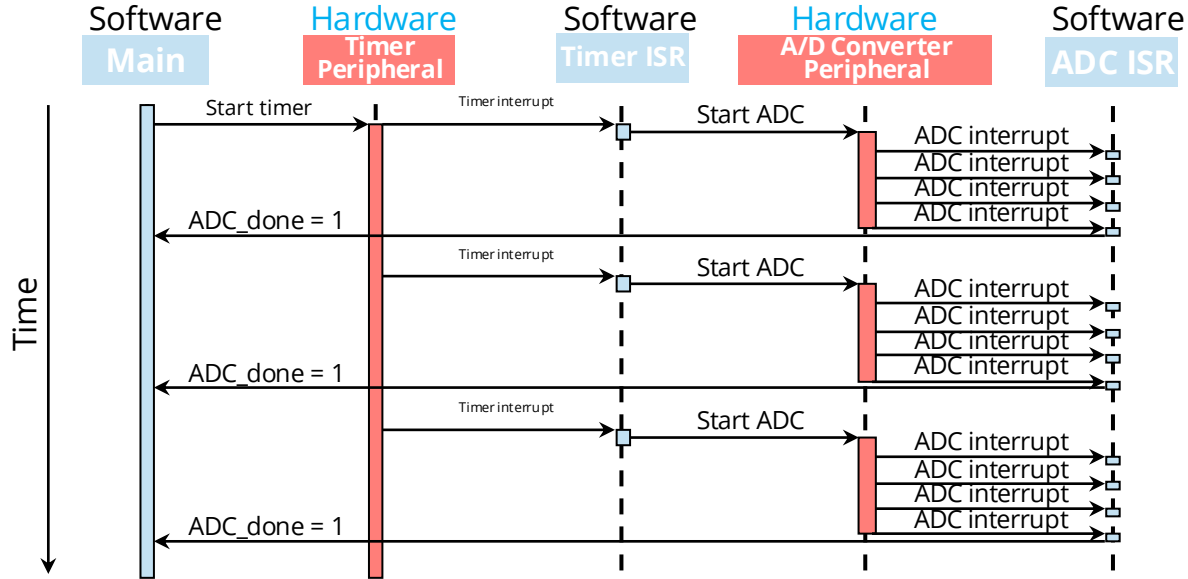
Actual relationship between depth and pressure

Concurrency through peripherals

- Things are happening **concurrently** while the CPU is executing instructions:
 - Peripherals continue working while the CPU executes application code.
- Peripherals use interrupts to notify the CPU about events
 - An interrupt transfers control to a short service routine
 - The routine records the event or moves data
 - The main program resumes after the urgent work is complete



Interrupt-driven execution



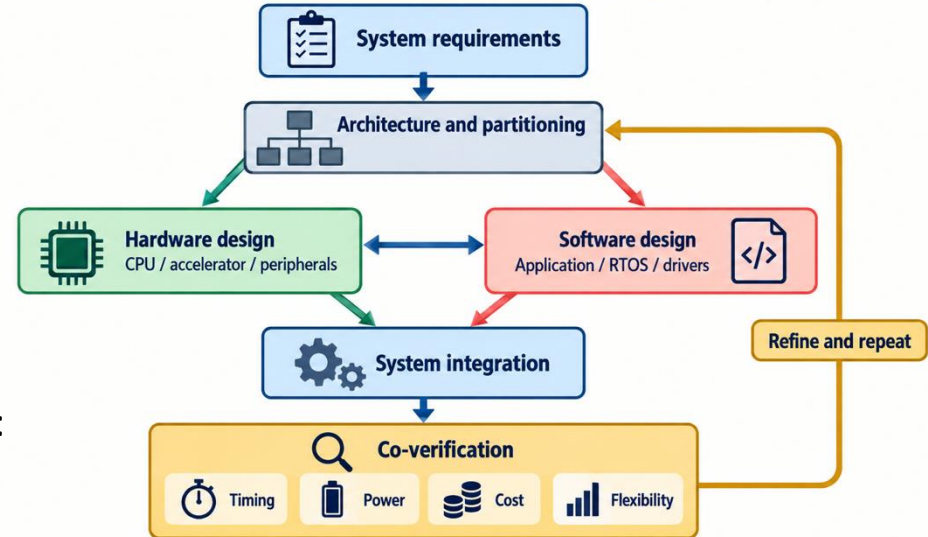
Embedded Software

- Programming language
 - Programmed in **C** rather than Java
 - smaller and faster code, so less expensive MCU
 - Some performance-critical code may be in **assembly language**
- Operating system
 - Typically, no OS
 - instead a **simple scheduler**
 - or even just interrupts + main code (**foreground/background system**)
 - If OS is used, likely to be an embedded RTOS

Hardware and Software Co-design Model

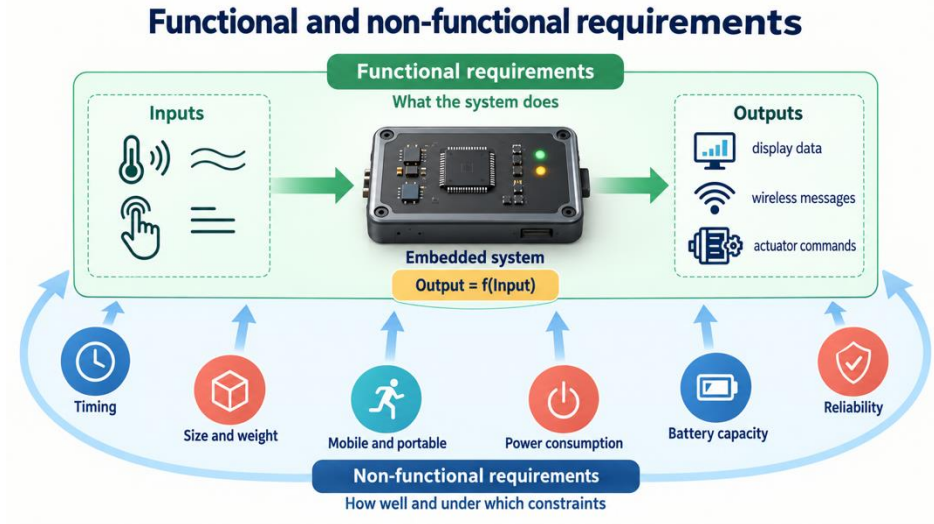
- Commonly both the hardware and the software for an embedded system are developed in parallel.
 - If performance is needed:
 - take advantage of special hardware features.
 - Push things to the software layer if functionality can be achieved in software:
 - Reduces overall hardware complexity and cost.

Hardware-software co-design



Functional vs. Non-functional Requirements

- Functional requirements:
 - Output as a function of input.
- Non-functional requirements:
 - Time required to compute output,
 - Size, weight (mobile and portable)
 - Power consumption (battery capacity)
 - Reliability



Functional vs. Non-functional Requirements

- Functionality is a must but embedded applications **should meet** non-functional requirements as well.
 - Performance goal: the program **must meet its deadline**
 - **Deadline:** the time at which a computation must be finished.
- For example, TV broadcast involves digital signal processing and cannot tolerate delays.
 - If the program does not produce the required output by the deadline:
 - The system **does not work properly** even if the output that it eventually produces is functionally correct.

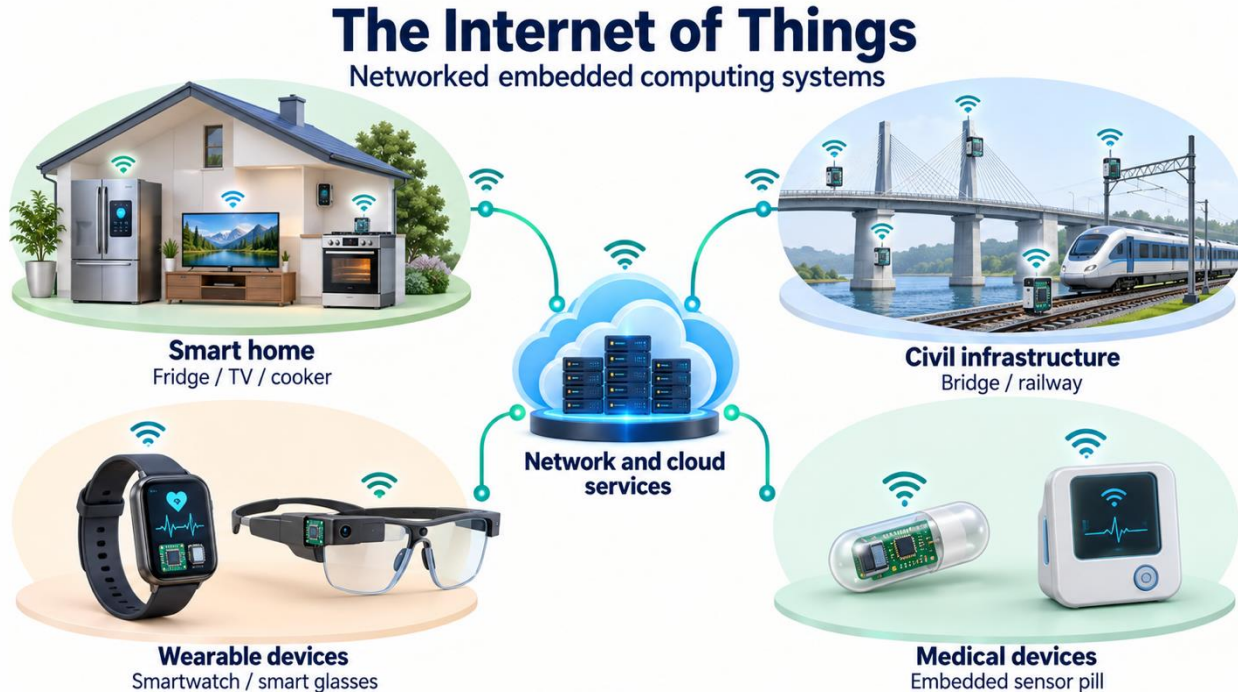
Functional vs. Non-functional Requirements

- Functionality is a must but embedded applications **should meet** non-functional requirements as well.
 - Performance goal: the program **must meet its deadline**
 - **Deadline:** the time at which a computation must be finished.
- For example, TV broadcast involves digital signal processing and cannot tolerate delays.
 - If the program does not produce the required output by the deadline:
 - The system **does not work properly** even if the output that it eventually produces is functionally correct.

We can use a digital signal processor (DSP), which has specialized arithmetic units to perform complex calculations extremely fast in real time.

The Internet of Things (IoT)

- Embedded devices **exchange data** and coordinate with software services across the physical world.



IoT device examples

- Different products share one pattern: sensing, local computation, **communication**, and action.



THERMOSTAT



WEARABLE



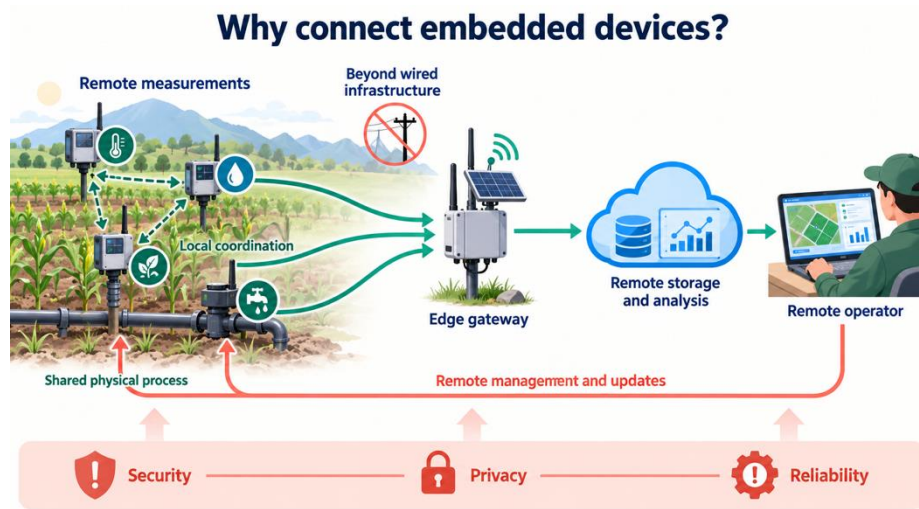
VOICE



SENSOR NODE

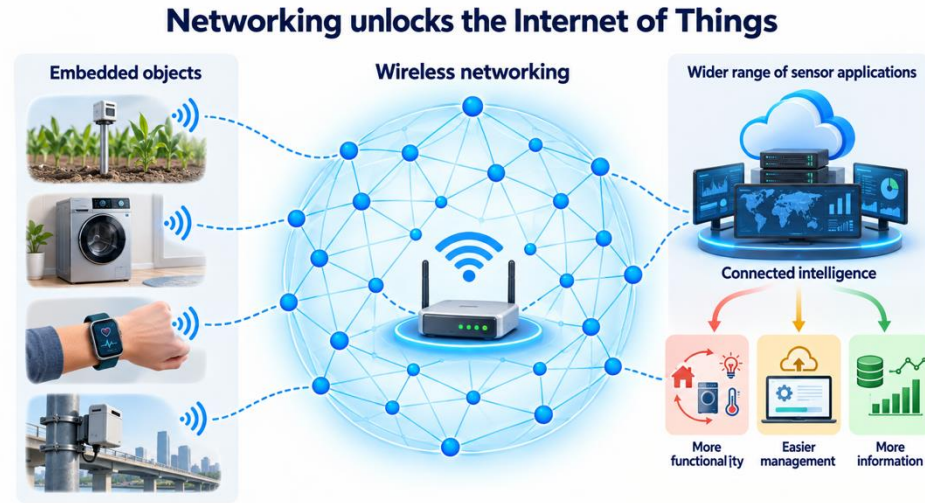
Why connect embedded devices?

- Collect measurements beyond the reach of wired infrastructure
- Manage devices and update behavior remotely
- Coordinate devices that share a physical process
- Combine local sensing with remote storage and analysis
- Expose new security, privacy, and reliability risks



Wireless Networking

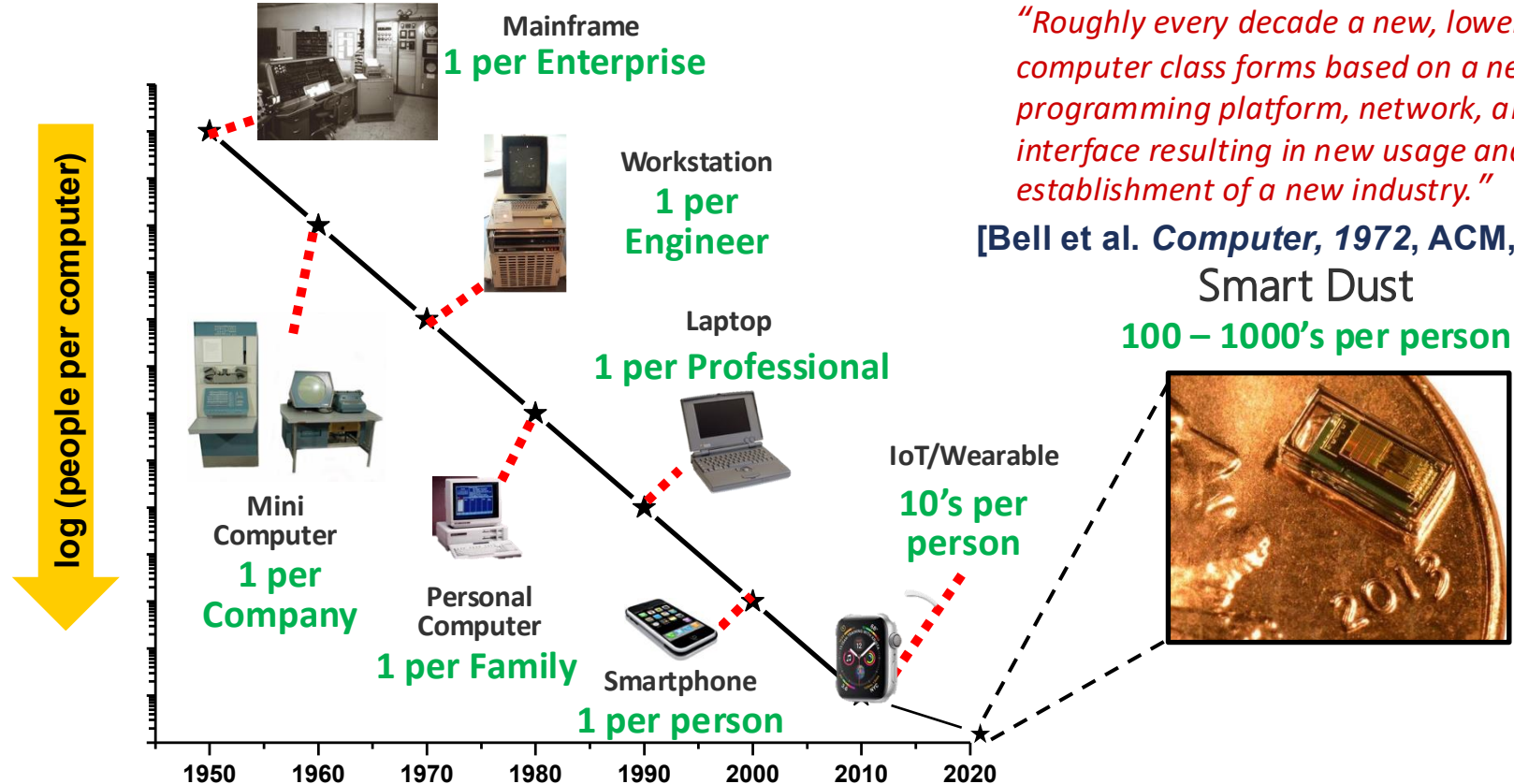
- Networking is a critical component of an IoT system
 - Allows a much wider range of sensor applications.
- Why IoT?
 - Items can have more functionality and become more intelligent
 - Items can be managed more easily
 - More information becomes available



Trillion embedded devices will be deployed soon

"Roughly every decade a new, lower priced computer class forms based on a new programming platform, network, and interface resulting in new usage and the establishment of a new industry."

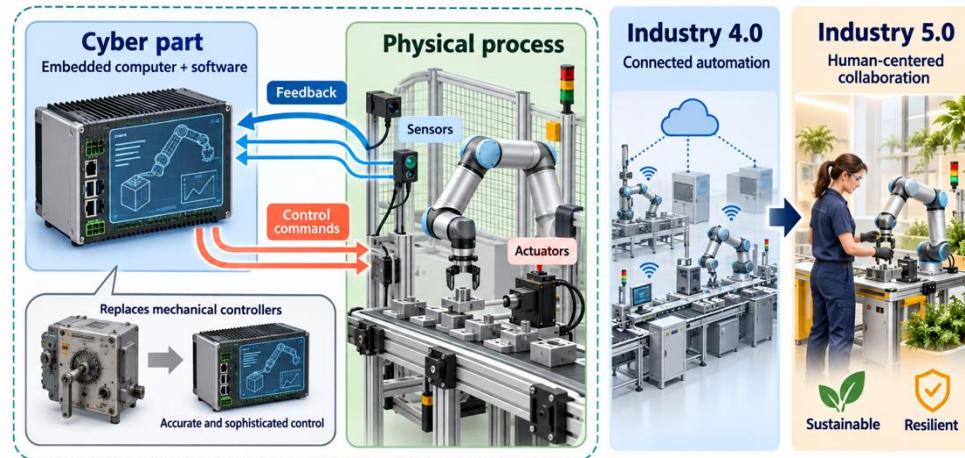
[Bell et al. *Computer*, 1972, ACM, 2008]



Cyber-physical Systems

- Combines physical devices with computers that control the device.
- The embedded computer is the **cyber part**
 - replaces mechanical controllers: more accurate and more sophisticated control
 - monitors and controls the physical processes with **feedback loops**

Cyber-physical systems



Summary

- An embedded system is built for a specific application.
 - Hardware and software components.
- Embedded systems pose many design challenges:
 - Functional and non-functional requirements
 - design time, deadlines, power, cost, etc.
- In real-time systems, timing correctness is just as important as functional or logical correctness.